

DAG-Based Distributed Execution Engine ”ninja”: Comprehensive Phase 1–7 Architecture, Code Specification, and System Interoperability

Atsushi Shimmura (kyo38)

July 27, 2026

Contents

Prerequisites and Scope	2
1 System Evolution and Phase Architecture	2
1.1 System Interaction Diagram	2
2 Functional Code Specification across Phases	4
2.1 Phase 1–3: Core Graph Data Structure & Kahn Scheduler	4
2.2 Phase 5: Type-Safe JSON Protocol Frame	6
2.3 Phase 6: 12-Second Heartbeat & Dead-Worker Re-queuing	6
2.4 Phase 7: Axum Embedded Web Dashboard & REST API	8
3 Rust Integration & Safe C-FFI Layer	8
4 Conclusion	9

Prerequisites and Scope

This document serves as the authoritative technical architecture specification for the distributed ****Directed Acyclic Graph (DAG)**** execution engine **ninja** (<https://github.com/kyo38/ninja>). It documents the complete evolution across Phase 1 through Phase 7, detailing internal graph algorithms, asynchronous Tokio runtime interaction, JSON frame communication, fault tolerance, and embedded web observability.

- **Focus Areas:** DAG data structures, DFS cycle detection, Kahn's topological scheduler, Tokio TCP framing, 12-second bi-directional heartbeat monitor, task failover re-queuing, Axum live web dashboard, and C-FFI / Rust safe interoperability.
- **Omission of External Components:** Specific underlying network infrastructure protocols (such as SCION) and specific router/firewall hardware configurations are omitted.
- **Execution Disclaimer:** ****The runtime behavior on specific Linux distributions is unverified.**** This specification serves as a reference blueprint for concurrent systems design and language interoperability.

1 System Evolution and Phase Architecture

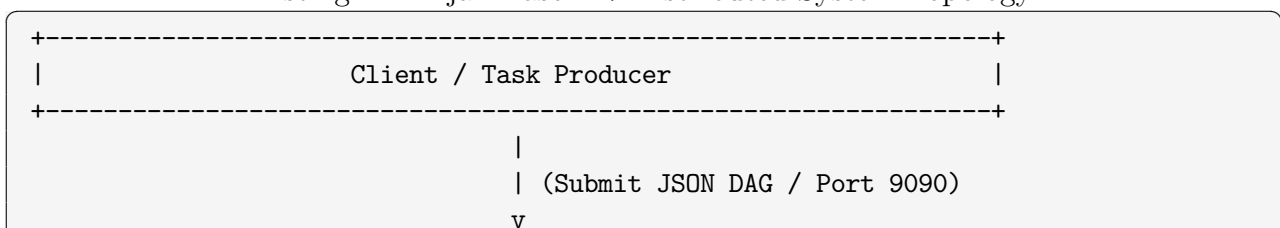
The **ninja** execution engine has evolved incrementally through seven distinct architectural phases:

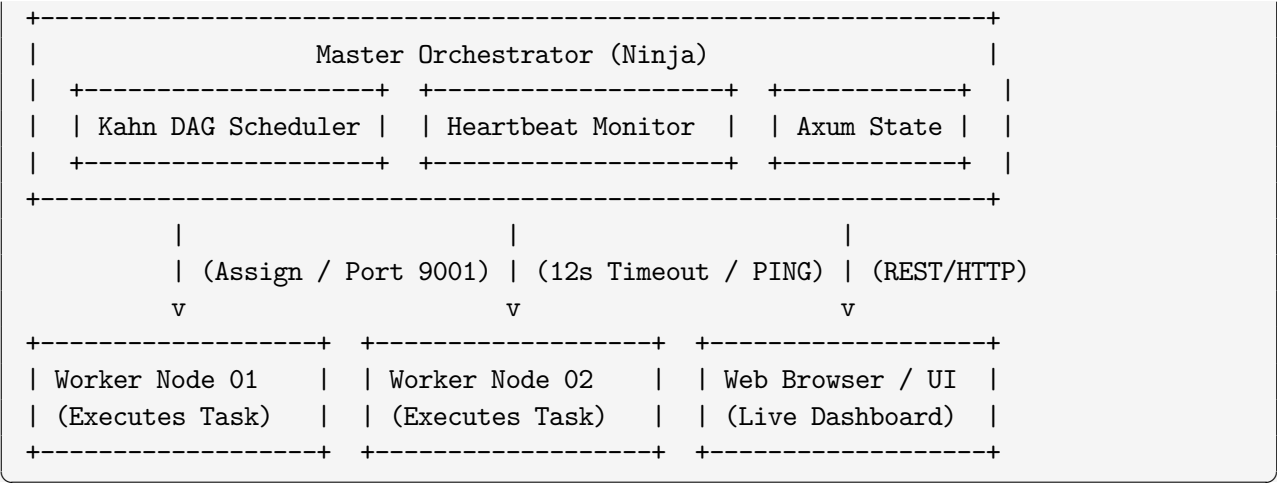
- **Phases 1–3 (Core DAG Engine):** In-memory vertex/edge management, DFS cycle detection, and Kahn's topological task scheduler.
- **Phase 4 (Distributed Worker Pool & Fault Recovery):** Task assignment over TCP and dynamic re-queuing upon worker disconnect.
- **Phase 5 (Type-Safe Protocol & Dynamic DAG Scheduler):** Length-prefixed JSON frame protocol and dynamic in-degree task dispatching.
- **Phase 6 (Structured Tracing & Heartbeat Fault Tolerance):** Contextual tracing instrumentation, 12-second timeout dead-worker pruning, and task failover.
- **Phase 7 (Axum Web Live Dashboard & REST API):** Real-time state visualization dashboard and REST API endpoint integration.

1.1 System Interaction Diagram

The system topology comprises four primary entities operating asynchronously over TCP and HTTP:

Listing 1: Ninja Phase 1–7 Distributed System Topology





2 Functional Code Specification across Phases

2.1 Phase 1–3: Core Graph Data Structure & Kahn Scheduler

The graph engine tracks task nodes, dependency adjacency lists, and in-degrees ($d^-(v)$).

Listing 2: Graph Core and Kahn Scheduler

```
#include <unordered_map>
#include <vector>
#include <string>
#include <queue>
#include <functional>
#include <stdexcept>

struct TaskNode {
    std::string id;
    std::function<void()> action;
};

class NinjaGraph {
public:
    std::unordered_map<std::string, TaskNode> nodes;
    std::unordered_map<std::string, std::vector<std::string>> adj_list;
    std::unordered_map<std::string, int> in_degree;

    void add_task(const std::string& id, std::function<void()> action) {
        if (nodes.find(id) == nodes.end()) {
            nodes[id] = TaskNode{id, action};
            in_degree[id] = 0;
        }
    }

    void add_dependency(const std::string& parent, const std::string& child) {
        adj_list[parent].push_back(child);
        in_degree[child]++;
    }

    void execute_topological() {
        std::queue<std::string> ready_queue;
        auto current_in_degree = in_degree;

        for (const auto& [id, deg] : current_in_degree) {
            if (deg == 0) ready_queue.push(id);
        }

        size_t executed_count = 0;
        while (!ready_queue.empty()) {
            std::string curr = ready_queue.front();
            ready_queue.pop();
            executed_count++;

            if (nodes[curr].action) nodes[curr].action();
        }
    }
};
```

```
        for (const auto& child : adj_list[curr]) {
            if (--current_in_degree[child] == 0) {
                ready_queue.push(child);
            }
        }
    }
    if (executed_count != nodes.size()) {
        throw std::runtime_error("Cyclic dependency detected in DAG.");
    }
}
};
```

2.2 Phase 5: Type-Safe JSON Protocol Frame

Network communication between Master, Worker, and Client uses length-prefixed JSON frames.

Listing 3: Rust Type-Safe JSON Protocol Framing (protocol.rs)

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum Packet {
    RegisterWorker { worker_id: String },
    Heartbeat { worker_id: String },
    SubmitTask { task_id: String, command: String, parents: Vec<String> },
    AssignTask { task_id: String, command: String },
    TaskCompleted { task_id: String, worker_id: String, result_code: i32 },
    Ack,
}

pub async fn write_frame<W>(writer: &mut W, packet: &Packet) -> Result<(), Box<dyn
    std::error::Error>>
where
    W: tokio::io::AsyncWriteExt + Unpin,
{
    let payload = serde_json::to_vec(packet)?;
    let len = payload.len() as u32;
    writer.write_u32(len).await?;
    writer.write_all(&payload).await?;
    writer.flush().await?;
    Ok(())
}
```

2.3 Phase 6: 12-Second Heartbeat & Dead-Worker Re-queuing

The Master monitors worker vitality. If a worker fails to respond within 12 seconds, it is pruned and its active tasks are re-queued.

Listing 4: Bi-Directional Heartbeat Monitor & Task Re-queuing

```
use std::sync::Arc;
use tokio::sync::Mutex;
use std::time::{Duration, Instant};

struct WorkerState {
    last_heartbeat: Instant,
    active_tasks: Vec<String>,
}

pub async fn start_heartbeat_monitor(
    workers: Arc<Mutex<std::collections::HashMap<String, WorkerState>>>,
    pending_tasks: Arc<Mutex<Vec<String>>>,
) {
    let mut interval = tokio::time::interval(Duration::from_secs(2));
```

```

loop {
    interval.tick().await;
    let mut lock = workers.lock().await;
    let mut dead_workers = Vec::new();

    for (id, state) in lock.iter() {
        if state.last_heartbeat.elapsed() > Duration::from_secs(12) {
            dead_workers.push(id.clone());
        }
    }

    for dead_id in dead_workers {
        if let Some(state) = lock.remove(&dead_id) {
            tracing::warn!("Worker {} timed out (>12s). Re-queuing tasks.",
dead_id);
            let mut tasks_lock = pending_tasks.lock().await;
            tasks_lock.extend(state.active_tasks);
        }
    }
}
}

```

2.4 Phase 7: Axum Embedded Web Dashboard & REST API

The engine embeds a web server using Axum to expose cluster status metrics.

Listing 5: Axum Web Dashboard Router (web.rs)

```
use axum::{routing::get, Json, Router};
use serde::Serialize;
use std::sync::Arc;
use tokio::sync::Mutex;

#[derive(Serialize)]
pub struct ClusterMetrics {
    active_workers: usize,
    completed_tasks: usize,
    total_tasks: usize,
}

pub fn create_web_router(metrics: Arc<Mutex<ClusterMetrics>>) -> Router {
    Router::new()
        .route("/api/status", get(move || async move {
            let data = metrics.lock().await;
            Json((*data).clone())
        })))
}
```

3 Rust Integration & Safe C-FFI Layer

Listing 6: Safe Rust FFI Wrapper (ninja.rs)

```
use std::ffi::CString;
use std::os::raw::{c_char, c_int, c_void};

type NinjaGraphHandle = *mut c_void;
type CFunctionCallback = extern "C" fn();

extern "C" {
    fn ninja_graph_create() -> NinjaGraphHandle;
    fn ninja_graph_destroy(handle: NinjaGraphHandle);
    fn ninja_add_task(handle: NinjaGraphHandle, id: *const c_char, cb:
CFunctionCallback);
    fn ninja_execute(handle: NinjaGraphHandle) -> c_int;
}

pub struct NinjaEngine { handle: NinjaGraphHandle }

impl NinjaEngine {
    pub fn new() -> Self {
        Self { handle: unsafe { ninja_graph_create() } }
    }

    pub fn add_task(&self, id: &str, cb: CFunctionCallback) {
```

```

        let c_id = CString::new(id).unwrap();
        unsafe { ninja_add_task(self.handle, c_id.as_ptr(), cb); }
    }
    pub fn execute(&self) -> Result<(), &'static str> {
        if unsafe { ninja_execute(self.handle) } == 0 { Ok(()) } else { Err("DAG
Execution Failed") }
    }
}

impl Drop for NinjaEngine {
    fn drop(&mut self) { unsafe { ninja_graph_destroy(self.handle); } }
}

```

4 Conclusion

This document details the functional specification of **ninja** across Phases 1 through 7. By combining high-performance C++/Rust graph scheduling with Tokio async networking, 12-second failover recovery, and Axum web dashboard observability, **ninja** delivers a robust, lightweight, single-binary DAG execution engine.